

```
C:\Users\pc>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.email=arendon2330@gmail.com
user.name=arendon2330
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
gui.wmstate=normal
gui.geometry=893x435+26+26 175 196
```

```
C:\Users\pc>|
```

¿PARA QUE SIRVE EL GIT CONFIG --LIST?

R/= Este comando permite mostrar la configuración que Git tiene actualmente establecida. Puede mostrar datos como el nombre de usuario, correo electrónico y otras configuraciones.



ESP
LAA



100%

Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\pc>git config --global user.email
arendon2330@gmail.com

C:\Users\pc>|

¿Para que sirve el comando git config --global user.email?
Este comando permite consultar el correo electrónico que Git
utilizará de manera global para identificar los commits
realizados por el usuario



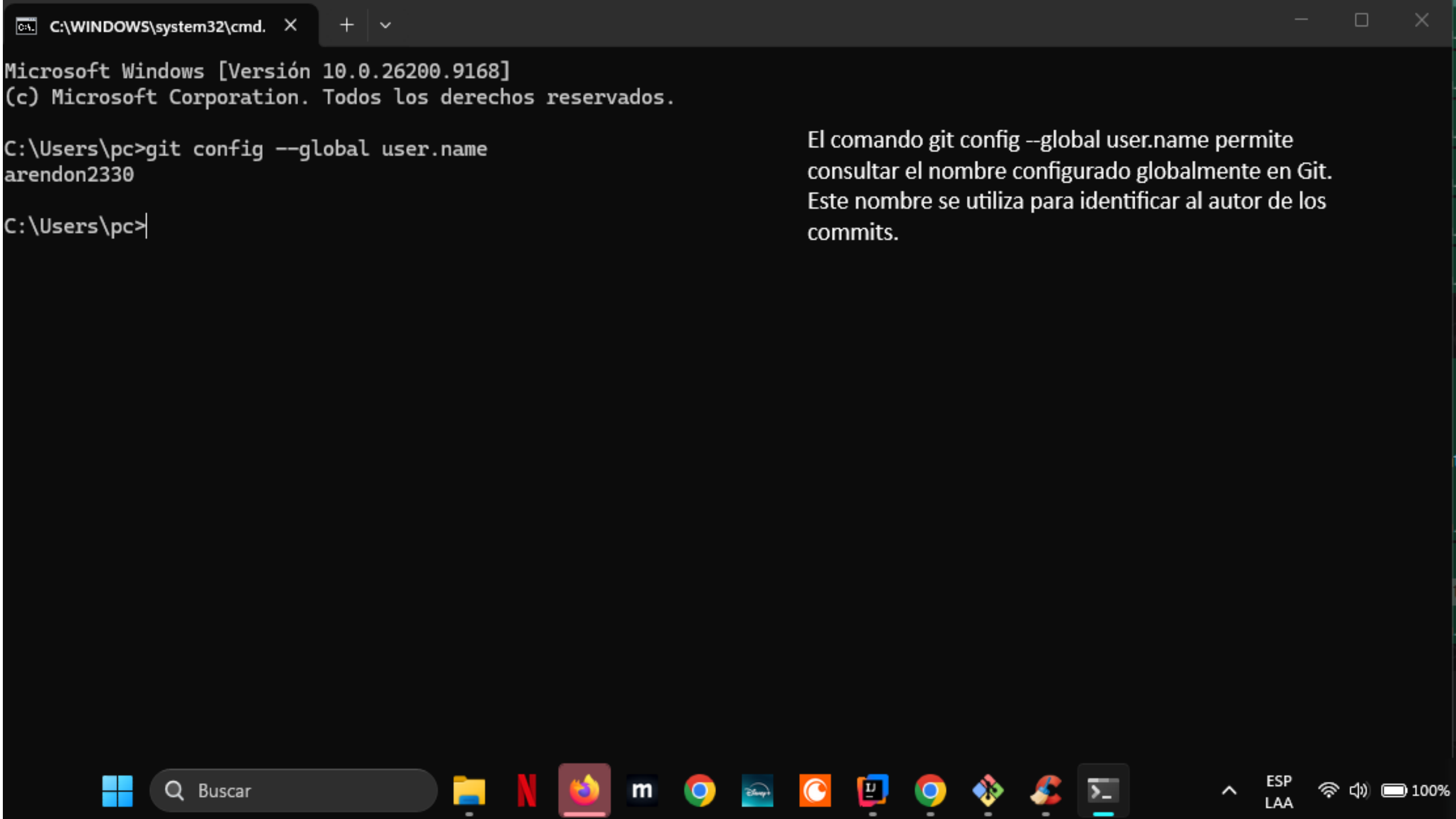
Buscar



ESP
LAA



100%



Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\pc>git config --global user.name
arendon2330

C:\Users\pc>

El comando git config --global user.name permite consultar el nombre configurado globalmente en Git. Este nombre se utiliza para identificar al autor de los commits.



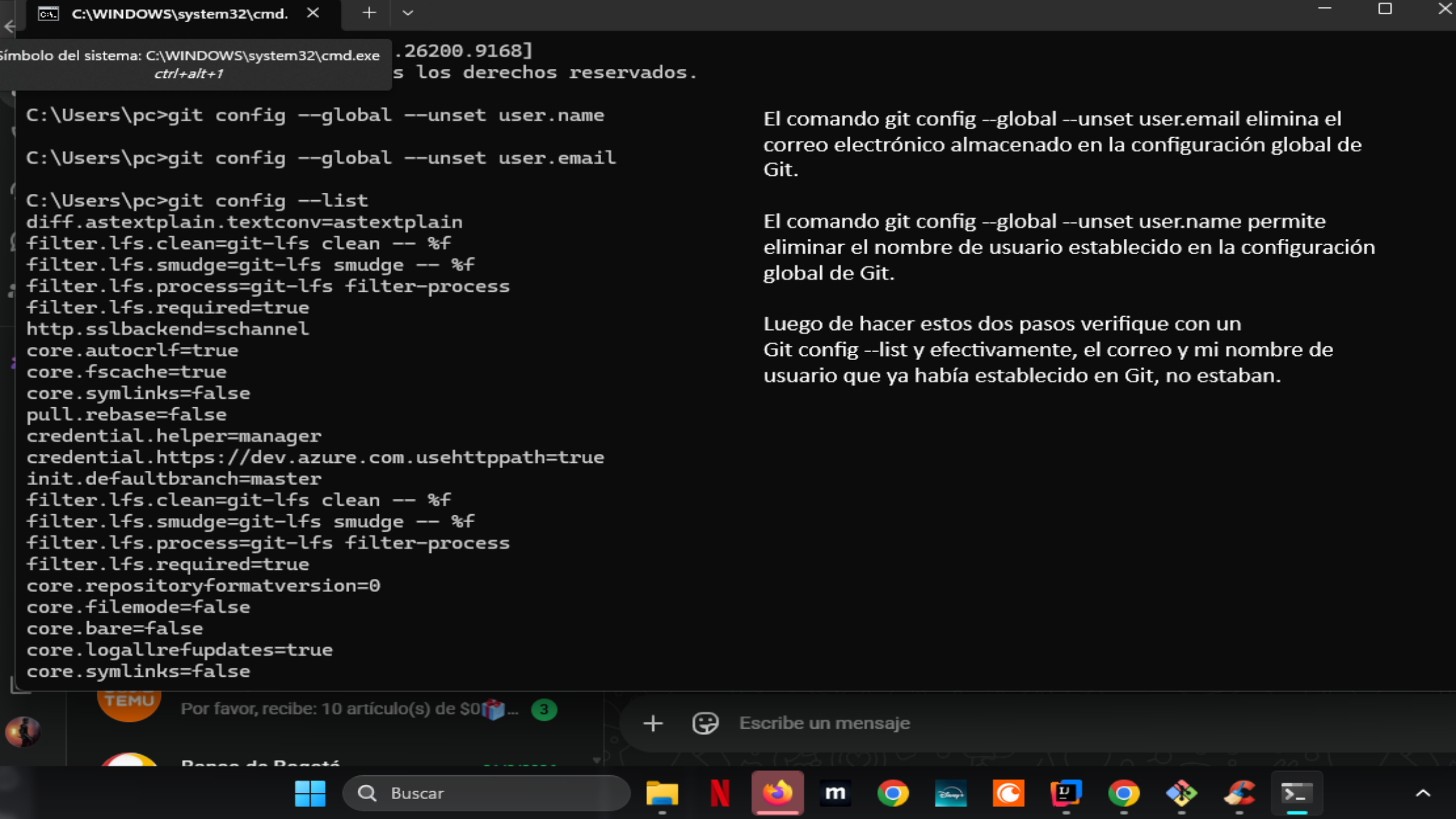
Buscar



ESP
LAA



100%



```
C:\Users\pc>git config --global --unset user.name
```

```
C:\Users\pc>git config --global --unset user.email
```

```
C:\Users\pc>git config --list
```

```
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=schannel  
core.autocrlf=true  
core.fscache=true  
core.symlinks=false  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
core.repositoryformatversion=0  
core.filemode=false  
core.bare=false  
core.logallrefupdates=true  
core.symlinks=false
```

El comando `git config --global --unset user.email` elimina el correo electrónico almacenado en la configuración global de Git.

El comando `git config --global --unset user.name` permite eliminar el nombre de usuario establecido en la configuración global de Git.

Luego de hacer estos dos pasos verifique con un `Git config --list` y efectivamente, el correo y mi nombre de usuario que ya había establecido en Git, no estaban.

```
Key@KEY MINGW64 ~ (main)
$ echo "# proyecto-3er-semester" >> README.md

Key@KEY MINGW64 ~ (main)
$ git init
Reinitialized existing Git repository in C:/Users/pc/.git/

Key@KEY MINGW64 ~ (main)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

Key@KEY MINGW64 ~ (main)
$ git commit -m "first commit"
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Enumerating cruft objects: 17074, done.
Traversing cruft objects: 17074, done.
Counting objects: 100% (17074/17074), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16684/16684), done.
Writing objects: 100% (17074/17074), done.
Total 17074 (delta 1685), reused 0 (delta 0), pack-reused 0 (from 0)
Removing duplicate objects: 100% (256/256), done.
[main (root-commit) a93e80c4] first commit
1 file changed, 1 insertion(+)
create mode 100644 README.md

Key@KEY MINGW64 ~ (main)
$ |
```

El comando **git init** inicializa una carpeta como un repositorio Git. A partir de ese momento Git puede comenzar a controlar y registrar los cambios realizados en los archivos del proyecto.



Buscar



ESP
LAA



45%

```
Key@KEY MINGW64 ~/proyecto-3er-semestre (main)
```

```
$ git add status.txt
```

```
warning: in the working copy of 'status.txt', LF will be replaced by CRLF the next time Git touches it
```

```
Key@KEY MINGW64 ~/proyecto-3er-semestre (main)
```

```
$
```

El comando git add . agrega todos los cambios de la carpeta actual a la zona de preparación. Estos cambios quedan listos para ser incluidos en el siguiente commit.
texto

```
Key@KEY MINGW64 ~/proyecto-3er-semestre (main)
```

```
$ git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
new file:   status.txt
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
proyecto-3er-semestre/
```

```
Key@KEY MINGW64 ~/proyecto-3er-semester (main)
$ git branch -r
origin/HEAD -> origin/main
origin/main
```

Descripción: El comando `git branch -r` muestra las ramas que existen en los repositorios remotos conocidos por el repositorio local.

```
key@KEY MINGW64 ~/proyecto-3er-semester (main)
$ git commit -m "Nuevo archivo status creado"
[main 149214f] Nuevo archivo status creado
1 file changed, 2 insertions(+)
create mode 100644 status.txt
```

Descripción: El comando `git commit -m` crea un nuevo registro en el historial de Git con los cambios que previamente fueron preparados mediante `git add`. El mensaje permite identificar qué cambios se realizaron.


```
Key@KEY MINGW64 ~/proyecto-3er-semester (main)  
$ git init proyecto-3er-semester  
Initialized empty Git repository in C:/Users/pc/proyecto-3er-semester/proyecto-3er-semester/.git/
```

Descripción: El comando git init inicializa una carpeta como un repositorio Git. A partir de ese momento Git puede comenzar a controlar y registrar los cambios realizados en los archivos del proyecto.

```
Key@KEY MINGW64 ~/proyecto-3er-semester (main)
$ nano status.txt

Key@KEY MINGW64 ~/proyecto-3er-semester (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    proyecto-3er-semester/
    status.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Descripción: El comando nano status.txt permite abrir o crear el archivo status.txt utilizando el editor de texto Nano. Es útil para crear o modificar archivos directamente desde la terminal antes de agregarlos y registrarlos en un repositorio Git.